

Nonlinear Programming

- Nonlinear programming (NLP) is a branch of mathematical optimization that deals with problems where the objective function or constraints are nonlinear.
- **LP vs NLP:** linear programming assumes constant marginal contributions and fixed input requirements. Nonlinear programming lets those change with the decisions we make.
 - How much fertilizer should a farmer apply? Yield gains diminish as fertilizer increases. Profit depends on a curved yield response.
 - How much should a firm invest in R&D? The marginal returns to R&D are decreasing, and the cost of R&D may increase with scale.
 - How should a producer allocate land across risky crops? The expected return is nonlinear in the allocation, and the risk depends on the covariance of returns across crops.

1 Unconstrained Optimization

1.1 Motivation

- Unconstrained optimization involves finding the maximum or minimum of a function without any restrictions on the variable values.
- Many economic models can first be understood in a frictionless, unconstrained environment before constraints are introduced.

1.1.1 Example

- **Profit maximization by a firm:**
A monopolist choosing output q to maximize profits:

$$\max_q \pi(q) = p(q)q - C(q)$$

where $p(q)$ is the inverse demand function and $C(q)$ is the cost function. If there are no capacity or policy restrictions, this is an unconstrained problem.

- Plot to build intuition:
 - Draw the demand curve and show how we are calculating the area of the rectangle under the curve (revenue). Assume linear $p(q) = a - bq$.
 - Then draw the cost curve. Assume constant marginal cost $C(q) = cq$.
 - Profit is the difference

If demand is linear and cost is linear, why isn't profit linear? As we sell another unit, what happens to revenue from the units we were already selling?

1.2 Mathematical Setup

The general unconstrained optimization problem can be written as:

$$\max_{x \in \mathbb{R}^n} f(x)$$

where $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is the objective function.

- The solution x^* is called an optimum if $f(x^*) \geq f(x)$ for all x in some neighborhood of x^* .

1.2.1 Assumptions for tractability

- $f(x)$ is continuous and differentiable at least once (so gradients exist).
- For stronger results, $f(x)$ is twice differentiable (so the Hessian exists).
- The function is well-behaved: bounded above (for maximization problems) or below (for minimization problems).

In economics, many standard functional forms (Cobb-Douglas, CES, quadratic, log-linear) satisfy these assumptions, making unconstrained optimization analytically tractable.

1.3 Univariate setup

If $f : \mathbb{R} \rightarrow \mathbb{R}$, the problem reduces to finding the optimal point x^* such that:

1.3.1 First-Order Condition

$$f'(x^*) = 0$$

What is the graphical intuition behind the first-order condition? Suppose our candidate max yields a positive FOC, which direction should we move?

1.3.2 Second-Order Condition

- $f''(x^*) < 0 \implies$ local maximum – evaluated at the critical point x^*
- $f''(x^*) > 0 \implies$ local minimum
- $f''(x^*) = 0 \implies$ inconclusive (could be inflection point, higher-order test needed; e.g., $f(x) = x^3$ at $x = 0$)

1.3.3 Example: A quadratic profit function

Return to our example. Set up the max problem

$$\pi(q) = aq - bq^2 - cq, \quad a, b, c > 0$$

The first-order condition is:

$$\pi'(q) = a - 2bq - c = 0 \implies q^* = \frac{a - c}{2b}$$

We can verify this is a local maximum by checking the second-order condition.

The second-order condition is:

$$\pi''(q) = -2b < 0 \quad (\text{since } b > 0)$$

1.3.4 Try yourself

Consider the profit-maximization problem:

$$\max_q \pi(q) = 100 - (q - 3)^4.$$

Find the profit-maximizing quantity and show that it is indeed a maximum.

1.4 Multivariate Optimization

If $f : \mathbb{R}^n \rightarrow \mathbb{R}$, the optimization problem is:

$$\max_{x \in \mathbb{R}^n} f(x_1, x_2, \dots, x_n)$$

1.4.1 Gradient and First-Order Conditions

$$\nabla f(x^*) = \begin{bmatrix} \frac{\partial f}{\partial x_1}(x^*) \\ \frac{\partial f}{\partial x_2}(x^*) \\ \vdots \\ \frac{\partial f}{\partial x_n}(x^*) \end{bmatrix} = 0$$

$\nabla f(x^*)$ is the **gradient vector** of first derivatives. It is also called the **Jacobian** in the multivariate case.

Suppose the gradient vector is $[0, 5]$. Have we found an optimum because one marginal return is zero? What small change would improve the objective?

1.4.2 Hessian and Second-Order Conditions

Let $H(x)$ denote the Hessian matrix of second derivatives:

$$H(x) = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \frac{\partial^2 f}{\partial x_1 \partial x_2} & \dots \\ \frac{\partial^2 f}{\partial x_2 \partial x_1} & \frac{\partial^2 f}{\partial x_2^2} & \dots \\ \vdots & \vdots & \ddots \end{bmatrix}$$

If the objective curves downward in each variable separately, is that enough to establish a maximum? Could changing both variables together improve it? Consider $f(x, y) = -x^2 - y^2 + 4xy$.

One way to classify the Hessian is by examining the **leading principal minors** — the determinants of the top-left $k \times k$ submatrices ($\det(H) = f_{xx}f_{yy} - (f_{xy})^2$, more than two vars gets complicated). Let H_k denote the $k \times k$ leading principal minor.

- If $H(x^*)$ is **negative definite** $\implies$ local maximum. The sequence of leading principal minors alternates in sign, starting with negative. $\det(H_1) < 0$, $\det(H_2) > 0$, etc.
- If $H(x^*)$ is **negative semi-definite** $\implies$ local maximum or saddle point. Determinant is non-negative. All odd-order leading principal minors are non-positive, and all even-order leading principal minors are non-negative. $\det(H_1) \leq 0$, $\det(H_2) \geq 0$, etc.
- If $H(x^*)$ is **positive definite** $\implies$ local minimum. All leading principal minors are positive. $\det(H_1) > 0$, $\det(H_2) > 0$, etc.
- If $H(x^*)$ is **positive semi-definite** $\implies$ local minimum or saddle point. All leading principal minors are non-negative. $\det(H_1) \geq 0$, $\det(H_2) \geq 0$, etc.
- If $H(x^*)$ is **indefinite** $\implies$ saddle point. The leading principal minors do not follow a consistent sign pattern.

1.5 Multivariate Example: Two-Variable Unconstrained Maximization

Consider the problem:

$$\max_{x,y} f(x, y) = 100x + 150y - x^2 - y^2 - xy$$

Step 1: First-Order Conditions

Compute the partial derivatives:

$$\frac{\partial f}{\partial x} = 100 - 2x - y$$

$$\frac{\partial f}{\partial y} = 150 - 2y - x$$

Set both equal to zero:

$$100 - 2x - y = 0$$

$$150 - x - 2y = 0$$

Solve the system:

From the first equation, $y = 100 - 2x$.

Substitute into the second:

$$150 - 2(100 - 2x) - x = 0 \implies 150 - 200 + 4x - x = 0 \implies 3x - 50 = 0.$$

Or Gaussian elimination, multiplying the first row by -2:

$$(4x - x) + (150 - 200) = 0 \implies 3x - 50 = 0.$$

So $x^* = \frac{50}{3} \approx 16.67$.

Then $y^* = 100 - 2(16.67) = 66.67$.

Step 2: Second-Order Conditions

Hessian matrix:

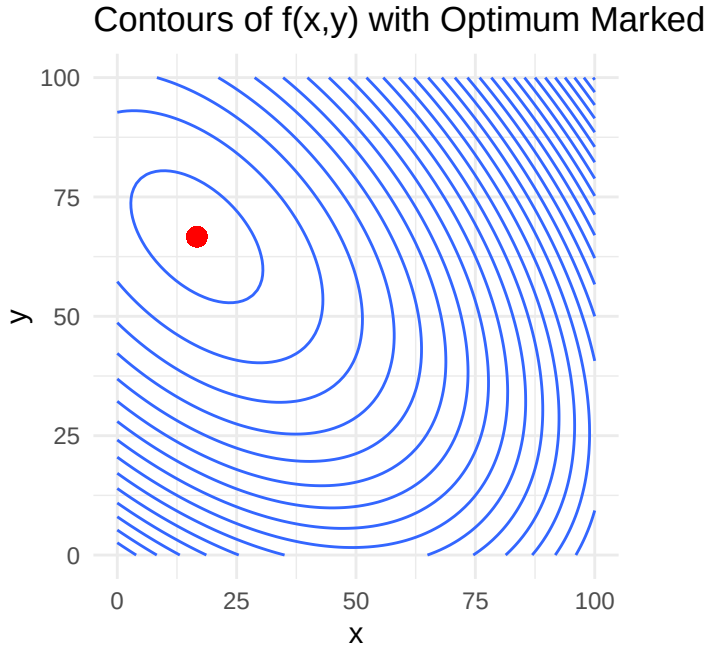
$$H = \begin{bmatrix} f_{xx} & f_{xy} \\ f_{yx} & f_{yy} \end{bmatrix} = \begin{bmatrix} -2 & -1 \\ -1 & -2 \end{bmatrix}$$

Compute its leading principal minors:

- Top-left element $|H_1| = -2 < 0$
- Determinant $|H_2| = (-2)(-2) - (-1)^2 = 4 - 1 = 3 > 0$

So H is **negative definite** $\rightarrow$ local maximum.

Step 3: Visualizing in R



1.6 Global vs. Local Optimality

Finding an optimum is not enough to guarantee that it is the **best possible solution**. We need to distinguish between **local** and **global** optima.

1.6.1 Local optimum

A point x^* is a **local maximum** if there exists a neighborhood $\mathcal{N}(x^*)$ such that:

$$f(x^*) \geq f(x) \quad \forall x \in \mathcal{N}(x^*).$$

Similarly, x^* is a **local minimum** if:

$$f(x^*) \leq f(x) \quad \forall x \in \mathcal{N}(x^*).$$

Local optima are determined by **first- and second-order conditions**:

- FOC: $\nabla f(x^*) = 0$
- SOC: Hessian conditions (positive/negative definiteness)

These tests, however, only ensure optimality relative to nearby points.

1.6.2 Global optimum

A point x^* is a **global maximum** if:

$$f(x^*) \geq f(x) \quad \forall x \in \mathbb{R}^n.$$

A point x^* is a **global minimum** if:

$$f(x^*) \leq f(x) \quad \forall x \in \mathbb{R}^n.$$

Global optimality requires stronger conditions about the **shape** of the objective function:

- If $f(x)$ is **strictly concave**, then any local maximum is also the global maximum.

- If $f(x)$ is **strictly convex**, then any local minimum is also the global minimum.

This is why assumptions of concavity and convexity are central in economics: they allow us to equate **marginal conditions** (first-order conditions) with **global results**.

1.6.3 Illustration

- A quadratic profit function

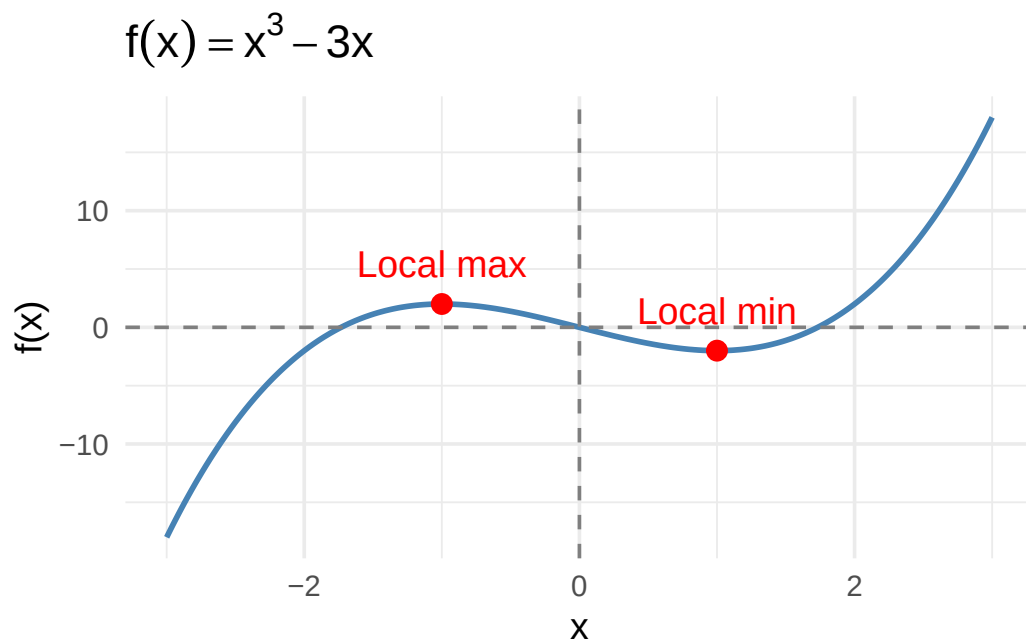
$$\pi(q) = aq - bq^2, \quad a, b > 0$$

is globally concave. The FOC $a - 2bq = 0$ and SOC $-2b < 0$, so π is globally concave.

- A non-concave function such as

$$f(x) = x^3 - 3x$$

has two local extrema ($x = -1, x = 1$). No global because as $x \rightarrow -\infty, f(x) \rightarrow -\infty$ and as $x \rightarrow \infty, f(x) \rightarrow \infty$. The FOC and SOC identify local points, but without concavity the global solution is not guaranteed.



1.6.4 Economic interpretation

- **Local optimality** corresponds to an agent being at a point where no *small deviation* is profitable.
- **Global optimality** ensures that the agent is making the *best possible choice* over the entire feasible set.

Economists typically assume concavity (utility, production, profit functions) to avoid the difficulty of distinguishing between local and global solutions. This ensures that solving the FOC is sufficient for finding the true economic optimum.

2 Numeric Foundations

- Numerical methods provide approximate solutions when analytical approaches are not feasible.
- They can work even when the function is not differentiable.
- Topics include rootfinding, gradient descent methods, and convergence criteria.

2.1 FOC to Rootfinding

Many optimization and equilibrium problems in economics reduce to **solving equations**.

$$\max_x f(x) \Rightarrow f'(x^*) = 0$$

this tells us what the solution looks like; it doesn't necessarily tell us how to find it.

For $f(x) = -(x-4)^2 + 10$, solving $f'(x) = 0$ analytically is trivial. But for more complicated functions, we may not be able to solve the FOC analytically. In these cases, we can use **numerical methods** to find a solution.

We want to find a solution x^* such that:

$$f(x^*) = 0$$

Where:

- $f : \mathbb{R}^n \rightarrow \mathbb{R}^n$
- $x^* \in \mathbb{R}^n$

This is the most general formulation. Economic examples include:

- Satisfying first-order conditions in maximization problems
- Finding market-clearing prices

i Quick digression on Fixed-Point Problems

A **fixed point** of a function $g(x)$ is a solution to:

$$x = g(x)$$

This is equivalent to rootfinding if we define $f(x) = x - g(x)$. Many economic models are naturally written this way:

- Value function iteration in dynamic programming
- Nash equilibria in games

Fixed-point methods often rely on contraction mappings, which guarantee convergence under certain conditions. More on this in dynamic programming.

2.2 Bisection Method

The **bisection method** is a simple and robust way to solve nonlinear equations of the form:

$$f(x) = 0$$

It is only applicable to **univariate** problems but serves as an intro to how numerical solvers work.

2.2.1 Requirements

To apply bisection:

- The function $f(x)$ must be **continuous** on the interval $[a, b]$

- The function must change sign over the interval: $f(a) \cdot f(b) < 0 \rightarrow$ This guarantees a root exists in $[a, b]$ by the **Intermediate Value Theorem**.

How does it guarantee this? either the a or b evaluates to negative and the other positive, so by the IVT there must be a point in between where it crosses zero.

2.2.2 Algorithm

1. Check that $f(a) \cdot f(b) < 0$. If not, bisection cannot proceed.
2. Compute the midpoint: $c = \frac{a+b}{2}$
3. Evaluate $f(c)$:
 - If $f(c) = 0$, stop — you found the root
 - If $f(c)$ has the same sign as $f(a)$, replace a with c
 - Otherwise, replace b with c
4. Repeat steps 2–3 until the interval is sufficiently small.

2.2.3 Example

Solve: $f(x) = x^2 - 2 = 0$

We know $x = \sqrt{2} \approx 1.4142$ is the root.

Start with $a = 1$, $b = 2$: $f(1) = -1$, $f(2) = 2$, so the root is bracketed.

Each iteration cuts the interval in half:

Iteration	a	b	$c = \frac{a+b}{2}$	$f(c)$
1	1.0	2.0	1.5	0.25
2	1.0	1.5	1.25	-0.4375
3	1.25	1.5	1.375	-0.109375
...	...	...	...	...

After several iterations, we get very close to the root.

2.2.4 Convergence Criteria

The bisection method illustrates an important question in all numerical methods:

When do we stop iterating?

There's no exact answer in most cases — so we stop when we're "close enough."

There are two common ways to define this:

1. Interval is small enough
 - We stop when: $|b - a| < \delta$. This ensures we're zoomed in tightly on the root. In bisection, this is the **primary** stopping rule.
2. Function value is small
 - Alternatively, we can stop when: $|f(c)| < \epsilon$. This ensures the function value at our current guess is nearly zero — a general-purpose convergence test used in many methods.

Why Both?

- In some methods, the updates $x^{(k+1)} - x^{(k)}$ may become small **even when the function value isn't** (e.g. at flat spots).

2.2.5 Multiple Roots

If the function has multiple roots in the interval, bisection will find one of them, but which one depends on the initial interval. If you need a specific root, you may need to adjust your interval accordingly. You could sample the domain to find intervals that bracket different roots.

Note also that if the function just touches zero (e.g., $f(x) = (x-1)^2$ at $x = 1$), bisection may fail since there is no sign change.

How might you improve the bisection method to find the root faster?

2.3 Newton's Method

Newton's method is a fast and widely used approach to optimizing nonlinear functions. Since optimization involves finding the roots of the first derivative, the method is applied to functions of the form:

$$f(x) = 0$$

It uses both the function **and its derivatives** to guide the search for the root. This makes it much faster than bisection or fixed-point iteration — but also more sensitive to where you start.

2.3.1 The update rule

The Newton update rule is:

$$x^{(k+1)} = x^{(k)} - \frac{f(x^{(k)})}{f'(x^{(k)})}$$

Take the local linear approximation of $f(x)$:

$$L(x) = f(x^{(k)}) + f'(x^{(k)})(x - x^{(k)})$$

We find the zero of this line by solving $L(x) = 0$, which gives:

$$x^{(k+1)} = x^{(k)} - \frac{f(x^{(k)})}{f'(x^{(k)})}$$

This is the heart of Newton's method: each step uses the slope of $f(x)$ to make a **smart jump** toward the root, rather than guessing blindly.

Intuition

Imagine standing on the curve of $f(x)$ at the point $(x^{(k)}, f(x^{(k)}))$. You draw the tangent line at that point. The next guess, $x^{(k+1)}$, is where this tangent line crosses the x-axis. This process is repeated, using the new point to draw a new tangent line, and so on, until you get sufficiently close to the root.

i Newton for optimization

To optimize an objective f , we instead solve its first-order condition, $f'(x) = 0$. Define $g(x) = f'(x)$ and apply the same Newton formula to g :

$$x_{k+1} = x_k - \frac{g(x_k)}{g'(x_k)} = x_k - \frac{f'(x_k)}{f''(x_k)}.$$

We optimize by finding the root of the derivative. This is the basis for **Newton-Raphson optimization**.

2.3.2 Newton's method assumptions

1. $f(x)$ is **differentiable** in the neighborhood of the root.
2. The derivative $f'(x)$ is **not zero** at the root (to avoid division by zero).
3. The initial guess $x^{(0)}$ is **sufficiently close** to the actual root for convergence.

4. $f(x)$ is sufficiently **smooth** (e.g., continuous second derivative) near the root to ensure good approximation by the tangent line.

2.3.3 Example

Let's revisit the problem $f(x) = x^2 - 2$

We have:

- $f(x) = x^2 - 2$
- $f'(x) = 2x$

Then Newton's update is:

$$x^{(k+1)} = x^{(k)} - \frac{x^{(k)^2} - 2}{2x^{(k)}}$$

Start with $x^{(0)} = 1$:

- $x^{(1)} = 1 - \frac{1^2 - 2}{2 \cdot 1} = 1 + 0.5 = 1.5$
- $x^{(2)} = 1.5 - \frac{1.5^2 - 2}{2 \cdot 1.5} \approx 1.4167$
- $x^{(3)} \approx 1.4142$

This converges very quickly to $\sqrt{2}$ — much faster than bisection.

```
library(ggplot2)

# Define the function and its derivative
f <- function(x) x^2 - 2
f_prime <- function(x) 2 * x

# Choose initial guess
x0 <- 1.0
x1 <- x0 - f(x0) / f_prime(x0) # first Newton update

# Build tangent line at x0
tangent <- function(x) f(x0) + f_prime(x0) * (x - x0)

# Create data for plotting
x_vals <- seq(0.5, 2, length.out = 300)
plot_df <- data.frame(x = x_vals, fx = f(x_vals), tangent = tangent(x_vals))

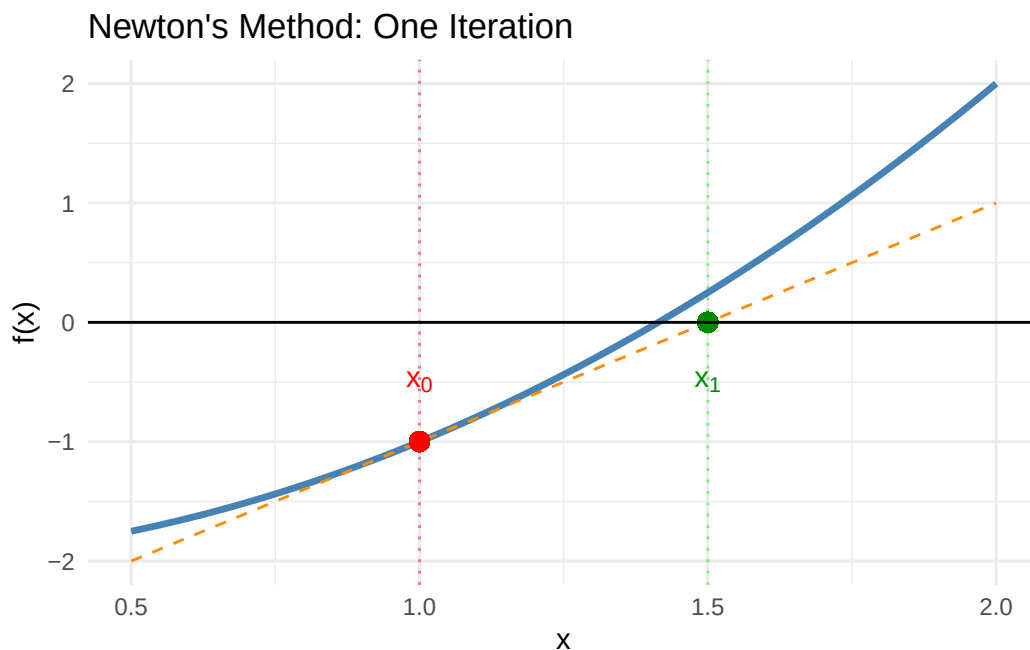
# Plot function and tangent
ggplot(plot_df, aes(x = x)) +
  geom_line(aes(y = fx), color = "steelblue", size = 1.2) +
  geom_line(aes(y = tangent), color = "darkorange", linetype = "dashed") +
  geom_vline(xintercept = x0, linetype = "dotted", color = "red", alpha=.5) +
  geom_vline(xintercept = x1, linetype = "dotted", color = "green", alpha=.5) +
  geom_point(aes(x = x0, y = f(x0)), color = "red", size = 3) +
  geom_point(aes(x = x1, y = 0), color = "green4", size = 3) +
  geom_hline(yintercept = 0, linetype = "solid") +
  annotate("text", x = x0, y = f(x0) + 0.5, label = "x[0]", parse = TRUE, color = "red") +
  annotate("text", x = x1, y = -0.5, label = "x[1]", parse = TRUE, color = "green4") +
  labs(
    title = "Newton's Method: One Iteration",
    y = "f(x)",
    x = "x"
```

```
) +  
theme_minimal()
```

Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
i Please use `linewidth` instead.

Warning in geom_point(aes(x = x0, y = f(x0)), color = "red", size = 3): All aesthetics have length 1, but `size` has length 3.
i Please consider using `annotate()` or provide this layer with data containing a single row.

Warning in geom_point(aes(x = x1, y = 0), color = "green4", size = 3): All aesthetics have length 1, but `size` has length 3.
i Please consider using `annotate()` or provide this layer with data containing a single row.



2.3.4 Convergence Properties

Newton's method has **quadratic convergence** when it works:

- The number of correct digits roughly **doubles** each iteration

But it can also:

- **Diverge** if $f'(x)$ is small or zero
- Converge to the **wrong root** if the starting point is poorly chosen
- Be unstable near points of inflection

2.3.5 Stopping Criteria

Same logic as before, but more care needed due to speed and sensitivity:

- Change in x : $|x^{(k+1)} - x^{(k)}| < \epsilon$
- Residual is small: $|f(x^{(k)})| < \epsilon$

In practice, it's common to use both.

2.3.6 When to Use Newton's Method

- Works best when you have access to an **analytic derivative** $f'(x)$
- Very effective for well-behaved, smooth functions
- Poor choice if $f'(x)$ is expensive to compute or not available

2.4 Quasi-Newton Methods

When the derivative $f'(x)$ is not available or too costly to compute, we can use **quasi-Newton methods**. These methods approximate the derivative using finite differences or other techniques.

2.4.1 Finite Difference Approximation

A simple way to approximate the derivative is using finite differences: $f'(x) \approx \frac{f(x^k) - f(x^{k-1})}{x^k - x^{k-1}}$

You can then plug this approximation into the Newton update formula: $x^{(k+1)} = x^{(k)} - \frac{f(x^{(k)})}{f'(x^{(k)})}$

This approach is called the **secant method** in one dimension. You can substitute the finite difference approximation for the derivative in the Newton update.

2.5 Gradient descent/ascent

What if I know the slope but don't want to calculate or use curvature?

Returning to optimization, Newton's method applied to the FOC uses $x_{k+1} = x_k - f'(x_k)/f''(x_k)$. **Gradient descent/ascent** uses only the slope of the objective, with a positive **learning rate (step size)** α_k :

$$\text{Minimization: } x_{k+1} = x_k - \alpha_k f'(x_k),$$

$$\text{Maximization: } x_{k+1} = x_k + \alpha_k f'(x_k).$$

The sign chooses downhill or uphill; α_k scales the move. The actual distance moved is $\alpha_k |f'(x_k)|$.

Use the same objective as before: maximize $f(x) = -(x-4)^2 + 10$, whose slope is $f'(x) = -2(x-4)$ and whose maximum is at $x^* = 4$. Starting from $x_0 = 0$, a constant step size gives $x_{k+1} = x_k - 2\alpha(x_k - 4)$:

- **Too small** ($\alpha = 0.05$): progress is slow.
- **Reasonable** ($\alpha = 0.4$): iterates converge quickly to 4.
- **Too large** ($\alpha = 1$): iterates overshoot and oscillate between 0 and 8.
- **Much too large** ($\alpha = 1.2$): oscillations grow and the iterates diverge.

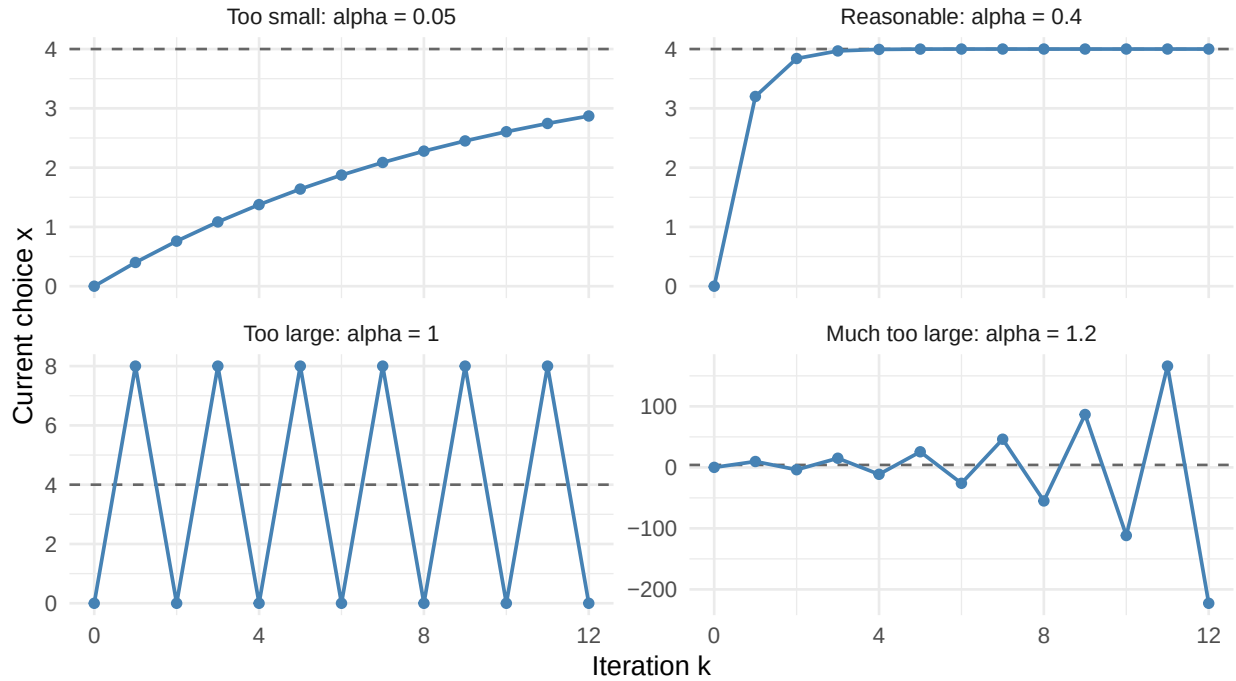


Figure 1: Gradient ascent on the same quadratic objective, starting at $x = 0$. The dashed line marks the maximizer $x = 4$. Note the different vertical scale in the divergence panel.

Line search: rather than choose α_k arbitrarily, an algorithm can search along the proposed direction for an acceptable step. For example, try a step and shrink it until it gives sufficient improvement: a decrease for minimization or an increase for maximization. This adapts the step size without requiring the second derivative or solving the line search exactly.

2.6 Summary: Comparing Rootfinding Algorithms

Method	Speed	Derivative Needed	Convergence Guarantee	Robustness	Common Use Cases
Bisection	Slow (linear)	No	Yes (if sign change)	Very robust	Bracketing roots in univariate problems
Newton's Method	Fast (quadratic)	Yes ($f'(x)$)	Local (if well-started)	Less robust	Solving FOCs, likelihood equations
Quasi-Newton Methods	Fast (superlinear)	No	Local (if well-started)	More robust than Newton	Large-scale optimization, machine learning

Choosing the Right Method

- Use **bisection** when you need guaranteed convergence and can bracket the root.
- Use **Newton's method** for fast convergence when you have a good initial guess and can compute derivatives.
- In practice, hybrid methods that combine these approaches are often employed to balance speed and robustness.

2.7 Multivariate optimization via rootfinding

2.7.1 Same idea, more decision variables

Move from $f(x)$ to $f(\mathbf{x})$, where $f : \mathbb{R}^n \rightarrow \mathbb{R}$. We still optimize **one objective**, such as profit or cost, but now choose n decision variables.

In one dimension, we optimize f by finding a root of $g(x) = f'(x)$. In multiple dimensions, the derivative becomes the **gradient**:

$$\nabla f(\mathbf{x}) = \begin{bmatrix} \partial f / \partial x_1 \\ \vdots \\ \partial f / \partial x_n \end{bmatrix}.$$

Define $\mathbf{g}(\mathbf{x}) = \nabla f(\mathbf{x})$. The first-order conditions for an interior optimum are $\mathbf{g}(\mathbf{x}^*) = \mathbf{0}$: every marginal change in the objective must be zero.

	One decision variable	Multiple decision variables
Objective	$f : \mathbb{R} \rightarrow \mathbb{R}$	$f : \mathbb{R}^n \rightarrow \mathbb{R}$
Function whose root we seek	$g(x) = f'(x)$	$\mathbf{g}(\mathbf{x}) = \nabla f(\mathbf{x})$
First-order condition	$g(x^*) = 0$	$\mathbf{g}(\mathbf{x}^*) = \mathbf{0}$

The objective is scalar-valued, but its gradient has n components. Finding a root of the gradient therefore means solving n **equations in n unknowns**. We are not trying to make the objective itself equal to zero.

2.7.2 Using the gradient to approach a stationary point

The gradient points in the direction of steepest increase.

For a fixed, small Euclidean step length, moving along the gradient gives the largest first-order increase in f . Moving against it gives the largest decrease. The one-variable gradient updates extend directly:

$$\begin{aligned} \text{Minimization: } \mathbf{x}_{k+1} &= \mathbf{x}_k - \alpha_k \nabla f(\mathbf{x}_k), \\ \text{Maximization: } \mathbf{x}_{k+1} &= \mathbf{x}_k + \alpha_k \nabla f(\mathbf{x}_k). \end{aligned}$$

The step size still matters: a direction that improves the objective locally can overshoot if we move too far.

Example: minimize the quadratic cost $f(x_1, x_2) = \frac{1}{2}(x_1^2 + 10x_2^2)$.

- Its gradient is $\mathbf{g}(\mathbf{x}) = [x_1, 10x_2]^\top$, so its unique root is $(0, 0)$, the cost minimizer.
- Cost rises more sharply in the x_2 direction, producing elliptical contours. Starting at $(4, 2)$ with $\alpha = 0.18$ gives $x_{1,k+1} = 0.82x_{1,k}$ and $x_{2,k+1} = -0.8x_{2,k}$: the second coordinate changes sign each step, while both approach zero.

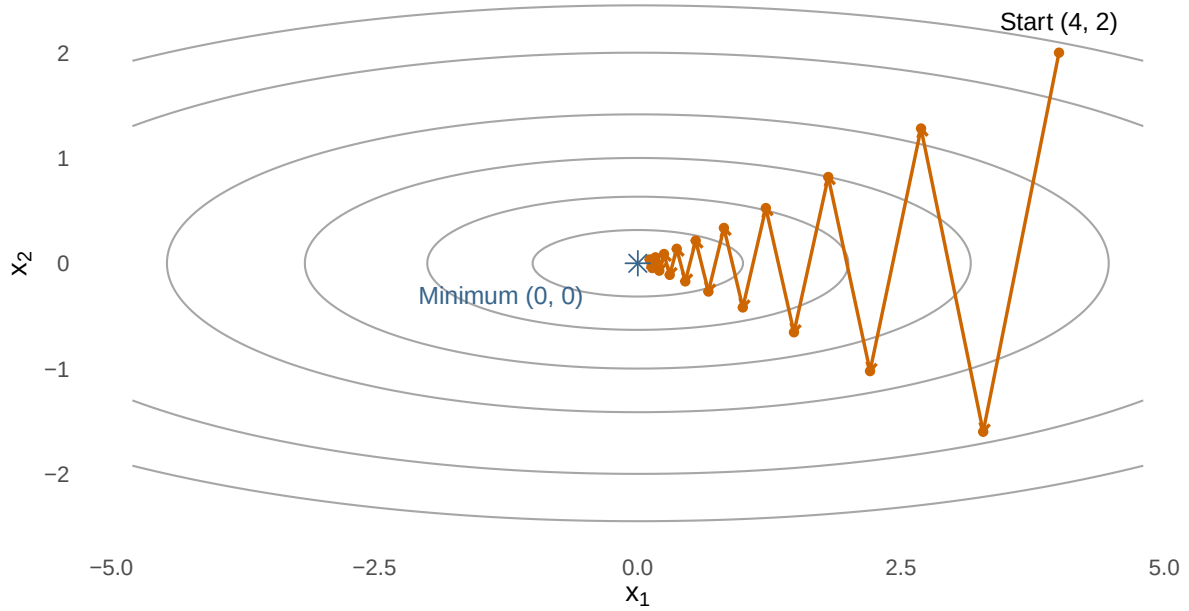


Figure 2: Gradient descent on elliptical cost contours. Each orange arrow is a step against the gradient. The iterates zig-zag toward the minimum at $(0, 0)$; a small gradient is the rootfinding target.

The same iterates on the cost surface show how the zig-zag moves down the bowl:

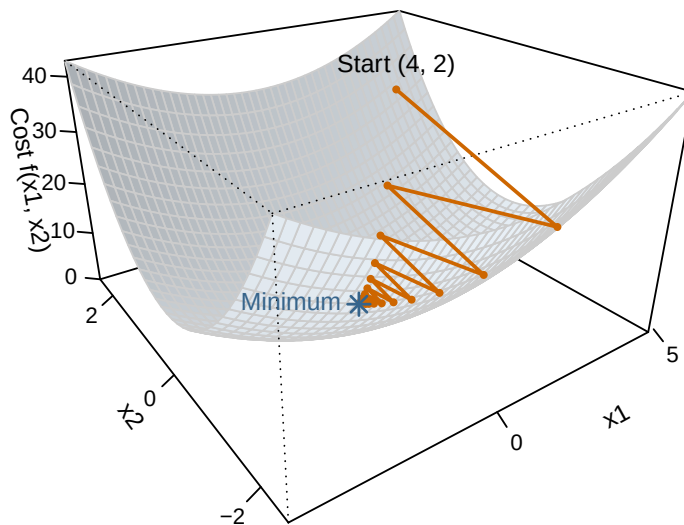


Figure 3: The same quadratic cost and gradient-descent iterates in three dimensions. Height is the objective value. Orange points are the iterates from the contour plot; segments connect successive iterates. The blue star marks the minimum.

2.7.3 Newton: find a root of the gradient

In one dimension, Newton uses g' to find a root of $g = f'$. In multiple dimensions, the derivative of $\mathbf{g} = \nabla f$ is the **Hessian**, $H(\mathbf{x}) = \nabla^2 f(\mathbf{x})$, whose entries are $H_{ij} = \partial^2 f / \partial x_i \partial x_j$. It describes how the marginal returns change as we change the decisions.

The same local linear approximation gives

$$\mathbf{g}(\mathbf{x}_k + \mathbf{d}) \approx \mathbf{g}(\mathbf{x}_k) + H(\mathbf{x}_k)\mathbf{d}.$$

Set this approximation equal to zero to obtain Newton's update (when the Hessian is nonsingular):

$$\underbrace{x_{k+1} = x_k - \frac{f'(x_k)}{f''(x_k)}}_{\text{One dimension}} \quad \longrightarrow \quad \underbrace{\mathbf{x}_{k+1} = \mathbf{x}_k - H(\mathbf{x}_k)^{-1} \nabla f(\mathbf{x}_k)}_{\text{Multiple dimensions}}.$$

- **Gradient:** which direction increases the objective most rapidly?
- **Hessian:** how steeply do marginal returns change in different directions?
- **Newton:** use both to choose a step toward zero marginal returns.

For the quadratic cost above, $H = \text{diag}(1, 10)$. Newton accounts for the different curvature in each direction and reaches $(0, 0)$ in one step, just as it did for our one-variable quadratic.

Both discussions target a **stationary point**. To conclude that it optimizes f , check the second-order conditions: a positive definite Hessian gives a strict local minimum, a negative definite Hessian a strict local maximum, and an indefinite Hessian a saddle point. A semidefinite Hessian is inconclusive. Away from the solution, Newton may still need a step-size or direction adjustment.

2.8 What actually goes wrong? (6 minutes)

Optimization software can return **an answer without returning the best answer**. Four issues matter in applied work:

- **Local versus global optima:** convergence to a local optimum does not establish global optimality. A small gradient and the right local curvature do not rule out a better solution elsewhere.
- **Starting values:** different $\mathbf{x}_0$ can lead to different solutions. Try several economically plausible starting values and compare objective values; agreement is useful evidence, not a proof of global optimality.
- **Scaling:** variables such as $x_1 \approx 0.001$ and $x_2 \approx 1,000,000$ can make step sizes, derivative calculations, and tolerances poorly balanced. Rescaling variables to comparable magnitudes can help without changing the underlying economic problem.
- **Stopping criteria:** “converged” usually means a numerical tolerance was satisfied, not that mathematics certified the solution. A tiny change in the parameters or objective can reflect stagnation; check the gradient or FOC residual as well.

Example: maximize $f(x) = -(x^2 - 1)^2 + 0.3x$. Run the same gradient-ascent rule, $x_{k+1} = x_k + 0.05f'(x_k)$, from $x_0 = -1.5$ and $x_0 = 1.5$. Both runs approach roots of $f'(x) = -4x(x^2 - 1) + 0.3$ with negative second derivatives, but the right-hand peak has a higher objective value.

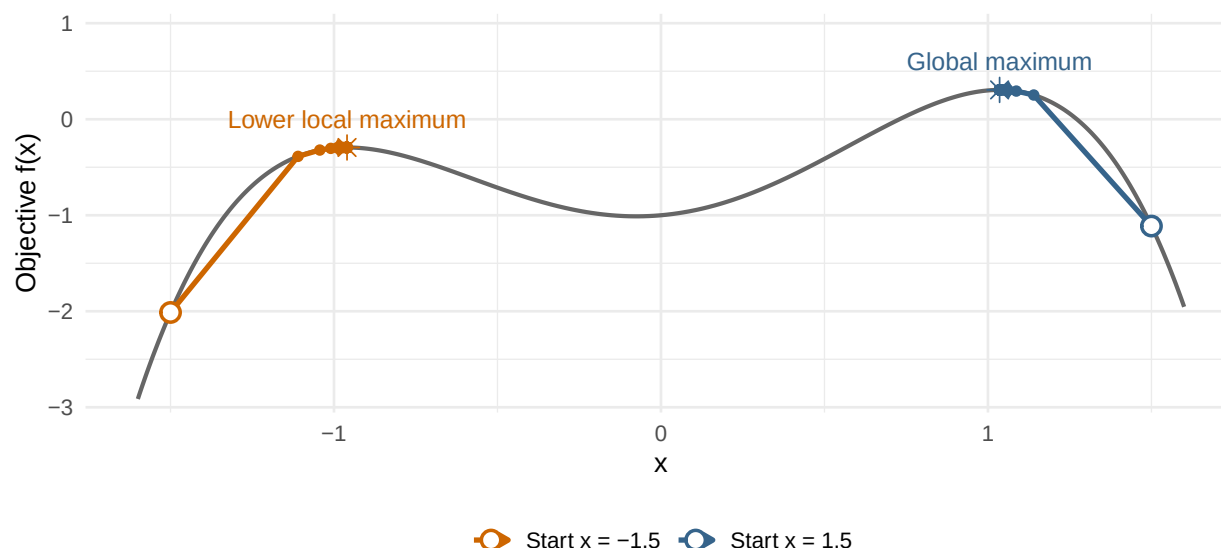


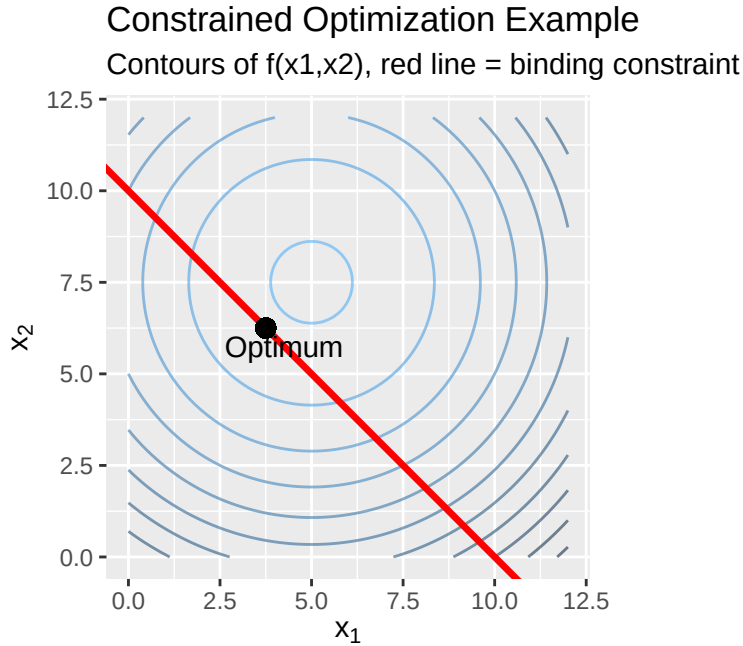
Figure 4: Same objective, algorithm, and step size; different starting values. Both runs converge to local maxima, but only the right-hand peak is the global maximum.

Before interpreting the economics, check the termination message, FOC residuals, local curvature, and sensitivity to starting values and scaling. Compare objective values across candidate solutions.

3 Constrained Optimization

Running Example

$$f(x_1, x_2) = 10x_1 + 15x_2 - x_1^2 - x_2^2$$
$$\text{s.t. } x_1 + x_2 \leq 10$$



3.1 Equality constraints: substitution and the Lagrangian

1. Solve through substitution: Substitute $x_2 = 10 - x_1$ and solve a one-variable problem.

$$f(x_1) = 10x_1 + 15(10 - x_1) - x_1^2 - (10 - x_1)^2$$

Substituting the constraint in reduces the 2 dimensional problem into 1, and imposes the constraint limiting the feasible set. FOC:

$$f'(x_1) = 10 - 15 - 2x_1 + 2(10 - x_1) = 15 - 4x_1 = 0 \implies x_1^* = \frac{15}{4} = 3.75$$

Then $x_2^* = 10 - x_1^* = 6.25$.

2. Solve through the Lagrangian (assume interior solution, so the constraint binds):

$$\mathcal{L} = f(x_1, x_2) + \lambda(10 - x_1 - x_2)$$

FOCs:

$$\mathcal{L}_{x_1} = 10 - 2x_1 - \lambda = 0$$

$$\mathcal{L}_{x_2} = 15 - 2x_2 - \lambda = 0$$

$$\mathcal{L}_{\lambda} = 10 - x_1 - x_2 = 0$$

Solving these three equations gives the same solution: $x_1^* = 3.75$, $x_2^* = 6.25$, and $\lambda^* = 2.5$.

Why do the two methods give the same condition? Along $x_2 = 10 - x_1$, increasing x_1 by one unit requires reducing x_2 by one unit. The chain rule accounts for both effects:

$$\frac{d}{dx_1}f(x_1, 10 - x_1) = f_{x_1} + f_{x_2} \frac{dx_2}{dx_1} = f_{x_1} - f_{x_2} = 0.$$

At the optimum, the marginal gain from increasing x_1 exactly offsets the marginal loss from reducing x_2 . The Lagrangian gives $f_{x_1} = \lambda$ and $f_{x_2} = \lambda$, which impose the same equality of marginal effects without substituting out a variable.

More generally, along $g(x_1, x_2) = b$, implicit differentiation gives $dx_2/dx_1 = -g_{x_1}/g_{x_2}$ when $g_{x_2} \neq 0$. The chain-rule condition becomes

$$f_{x_1} - f_{x_2} \frac{g_{x_1}}{g_{x_2}} = 0.$$

Defining $\lambda = f_{x_2}/g_{x_2}$ gives $f_{x_1} = \lambda g_{x_1}$ and $f_{x_2} = \lambda g_{x_2}$: together, $\nabla f = \lambda \nabla g$. Thus, the Lagrangian conditions encode the chain rule for movements that stay on the constraint.

3. Explain $\nabla f = \lambda \nabla g$: the objective has no first-order **improvement** along the constraint's tangent direction. In other words, you cannot increase f by moving up or down the constraint line. The gradient of f is **parallel** to the gradient of g at the optimum.
 - $\nabla g(x)$ points perpendicular to the constraint boundary (the “normal vector” to the feasible set).
 - $\nabla f(x)$ points in the unconstrained steepest ascent direction.
 - At the optimum on the boundary, these must line up — otherwise there would be a feasible direction in which f could still increase. One is just a rescaled version of the other, with the multiplier λ giving the scaling factor.
 - Emphasize that an equality multiplier is unrestricted in sign. Forcing the resource total above the unconstrained optimum can reduce value.

Checkpoint: At $b = 10$, $(x_1^*, x_2^*) = (3.75, 6.25)$, $f^* = 78.125$, and $\lambda^* = 2.5$. If the equality instead requires $b = 15$, the multiplier is -2.5 .

3.2 The Lagrangian and Karush-Kuhn-Tucker Conditions

The **Lagrangian** is

$$\mathcal{L}(x, \lambda) = f(x) + \lambda(b - g(x))$$

where λ are Lagrange multipliers (also called **shadow values**). The shadow price tells you the marginal value (in terms of the objective) of relaxing the resource constraint by a unit.

The **KKT conditions** are:

1. **Stationarity**

$$\frac{\partial \mathcal{L}}{\partial x} = \frac{\partial f(x^*)}{\partial x} - \lambda \frac{\partial g(x^*)}{\partial x} = 0$$

2. **Primal feasibility:** the solution x^* must lie in the feasible set. You cannot propose an “optimal” solution that violates the budget, land, time, or other resource limits.

$$g(x^*) \leq b$$

3. **Dual feasibility:** the Lagrange multipliers (shadow prices) must be nonnegative. Why? Because if a constraint is binding, the multiplier represents the increase in the objective from relaxing that constraint.

$$\lambda \geq 0$$

4. **Complementary slackness:** Either the constraint binds and holds with equality and the shadow price is nonnegative **or** the constraint does not bind and the shadow price is 0.

$$\lambda(b - g(x^*)) = 0$$

Can the constraint be binding and the shadow price be zero? Yes, if the constraint binds at the interior solution or if the constraint is redundant to another that also binds.

3.2.1 Second example

Use the chapter's $\max xy$ subject to $x + y \leq 10$, $x + 2y \leq 18$, and $x, y \geq 0$ (pp. 295–298). Use the KKT conditions to find the optimum and the shadow prices.

$$\mathcal{L}(x, y, \lambda_1, \lambda_2) = xy + \lambda_1(10 - x - y) + \lambda_2(18 - x - 2y)$$

KKT FOCs:

$$\frac{\partial \mathcal{L}}{\partial x} = y - \lambda_1 - \lambda_2 \leq 0, x \geq 0, x(y - \lambda_1 - \lambda_2) = 0$$

$$\frac{\partial \mathcal{L}}{\partial y} = x - \lambda_1 - 2\lambda_2 \leq 0, y \geq 0, y(x - \lambda_1 - 2\lambda_2) = 0$$

$$\frac{\partial \mathcal{L}}{\partial \lambda_1} = 10 - x - y \geq 0, \lambda_1 \geq 0, \lambda_1(10 - x - y) = 0$$

$$\frac{\partial \mathcal{L}}{\partial \lambda_2} = 18 - x - 2y \geq 0, \lambda_2 \geq 0, \lambda_2(18 - x - 2y) = 0$$

1. Both constraints bind: $x + y = 10$ and $x + 2y = 18$.

Just solve the two equations to get $x^* = 2$, $y^* = 8$.

Then plug the optimal x^* and y^* into the first two FOCs.

$$8 - \lambda_1 - \lambda_2 = 0 \implies \lambda_1 + \lambda_2 = 8$$

$$2 - \lambda_1 - 2\lambda_2 = 0 \implies \lambda_1 + 2\lambda_2 = 2$$

$$\implies \lambda_2 = -6, \lambda_1 = 14$$

Does this satisfy KKT conditions? dual feasibility

2. Only the second constraint binds: $x + y < 10$ and $x + 2y = 18$.

Because the first constraint does not bind, $\lambda_1 = 0$. Then plug into the FOCs:

$$y - 0 - \lambda_2 = 0 \implies y = \lambda_2$$

$$x - 0 - 2\lambda_2 = 0 \implies x = 2\lambda_2$$

$$18 - x - 2y = 0 \implies 18 - 2\lambda_2 - 2\lambda_2 = 0 \implies 4\lambda_2 = 18 \implies \lambda_2 = 4.5$$

Then $x^* = 2\lambda_2 = 9$, $y^* = \lambda_2 = 4.5$.

Does this satisfy KKT conditions? primal feasibility

3. Only the first constraint binds: $x + y = 10$ and $x + 2y < 18$.

Because the second constraint does not bind, $\lambda_2 = 0$. Then plug into the FOCs:

$$y - \lambda_1 - 0 = 0 \implies y = \lambda_1$$

$$x - \lambda_1 - 0 = 0 \implies x = \lambda_1$$

$$10 - x - y = 0 \implies 10 - \lambda_1 - \lambda_1 = 0 \implies 2\lambda_1 = 10 \implies \lambda_1 = 5$$

Then $x^* = \lambda_1 = 5$, $y^* = \lambda_1 = 5$.

Does this satisfy KKT conditions?

3.3 Bordered Hessian (Second Order Condition)

- The SOC's use the **bordered Hessian** to check the **curvature of f along feasible directions**.
- With constraints, we can't just look at the full Hessian. Some directions are **forbidden** (they would leave the feasible set).
- The bordered Hessian modifies the Hessian so that curvature is evaluated only along directions that *respect the constraints*.
- Imagine standing at the tangent plane of the feasible set at x^* . The bordered Hessian checks if f bends downward (for a max) or upward (for a min) *within that tangent plane*.

For one variable and one active constraint, the bordered Hessian is a 2×2 matrix:

$$H_B = \begin{bmatrix} 0 & -g_x(x^*) \\ -g_x(x^*) & \mathcal{L}_{xx}(x^*, \lambda^*) \end{bmatrix}.$$

(The minus sign comes from writing the constraint as $b - g(x) \geq 0$ in the Lagrangian.)

The determinant is

$$\det(H_B) = -[g'(x^*)]^2.$$

because $\mathcal{L}_{xx}(x^*, \lambda^*) * 0 = 0$.

- For a **constrained maximum** with one binding constraint, we require $\det(H_B) < 0$.
- For a **constrained minimum**, $\det(H_B) > 0$.

Suppose we have a vector x with n dimensions and q constraints.

$$H_B = \begin{bmatrix} 0 & \nabla_x g(x^*)^\top \\ \nabla_x g(x^*) & \nabla_{xx} \mathcal{L}(x^*, \lambda^*) \end{bmatrix}.$$

- The **sign pattern of the principal minors** of H_B gives the curvature test:
 - For a maximum: determinants alternate in sign, starting with $(-1)^q$ starting from $q + 1$.
 - For a minimum: determinants are all $(-1)^{q+r}$ for the r th minor.

In constrained optimization: You must “filter out” the directions that are not feasible. The **bordered Hessian** does that filtering — it's just a device to enforce that curvature is checked along the constraint surface.

3.3.1 Example

Let's check the SOC's for our earlier example.

$$\max_{x_1, x_2} f(x_1, x_2) = 10x_1 + 15x_2 - (x_1^2 + x_2^2) \quad \text{s.t. } x_1 + x_2 \leq 10$$

- Check the SOC's

$\nabla g(x) = [11]$ so the borders are 1. Then, $\nabla^2 f(x_1, x_2)$:

$$H_B = \begin{bmatrix} 0 & 1 & 1 \\ 1 & -2 & 0 \\ 1 & 0 & -2 \end{bmatrix}.$$

Starting from $q + 1 = 1 + 1 = 2$, must alternate sign starting with $(-1)^q = 1$:

$$H_{B_1} = \begin{bmatrix} 0 & 1 \\ 1 & -2 \end{bmatrix}$$

The determinant of H_{B_1} is $0 \cdot (-2) - 1 \cdot 1 = -1 < 0$.

and the full H_B :

$$H_B = \begin{bmatrix} 0 & 1 & 1 \\ 1 & -2 & 0 \\ 1 & 0 & -2 \end{bmatrix}.$$

The determinant of H_B is $0 \cdot (-2 \cdot -2 - 0 \cdot 0) - 1 \cdot (1 \cdot -2 - 0 \cdot 1) + 1 \cdot (1 \cdot 0 - (-2) \cdot 1) = 0 - (-2) + 2 = 4 > 0$.

3.4 Envelope Theorem

- Once you set up the Lagrangian, the effect of parameter changes on the optimal value of the objective is given by multipliers.
- Example: In a consumer problem, $\frac{\partial V}{\partial I} = \lambda$ (marginal utility of income).
- Economists use this all the time in comparative statics, welfare analysis, and derivations of demand.

3.5 Duality

- The problem you solved is the primal: maximize the objective subject to constraints.
- Every primal problem has a dual problem, where the variables are the multipliers (the λ 's).
- The dual is not just a mathematical trick — it has deep economic meaning: it's a price system that supports the allocation chosen in the primal.

The Lagrangian itself is the bridge:

$$\mathcal{L}(x, \lambda) = f(x) + \lambda(b - g(x)).$$

The dual function is

$$q(\lambda) = \max_x \mathcal{L}(x, \lambda).$$

The dual problem is

$$\min_{\lambda \geq 0} q(\lambda).$$

Economic interpretation: The dual chooses shadow prices λ so that the cost of resources (valued at λ) supports the primal allocation.

3.5.1 Example

The dual function is obtained by maximizing the Lagrangian over (x_1, x_2) for a given $\lambda \geq 0$:

$$q(\lambda) = \max_{x_1, x_2 \geq 0} \left[10x_1 + 15x_2 - x_1^2 - x_2^2 + \lambda(10 - x_1 - x_2) \right].$$

Stationarity conditions for the inner maximization:

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial x_1} = 10 - 2x_1 - \lambda = 0 & \Rightarrow x_1 = \frac{10 - \lambda}{2}, \\ \frac{\partial \mathcal{L}}{\partial x_2} = 15 - 2x_2 - \lambda = 0 & \Rightarrow x_2 = \frac{15 - \lambda}{2}. \end{aligned}$$

Plugging these $x_1(\lambda), x_2(\lambda)$ back into $\mathcal{L}$ gives the dual function. Compute each piece:

$$x_1 = \frac{10 - \lambda}{2}, x_2 = \frac{15 - \lambda}{2}.$$

$$q(\lambda) = 10 \frac{10 - \lambda}{2} - \left(\frac{10 - \lambda}{2} \right)^2 + 15 \frac{15 - \lambda}{2} - \left(\frac{15 - \lambda}{2} \right)^2 + \lambda(\lambda - 2.5).$$

The dual problem

$$\min_{\lambda \geq 0} q(\lambda).$$

Takeaways

- The dual problem says: find the resource price so that when agents maximize net benefit subject to that price, total demand for the resource just equals 10.
- The primal says: given 10 units of the resource, allocate across x_1 and x_2 to maximize net benefit.

Both yield the same value function, which is exactly the welfare-equilibrium duality economists care about.

4 Numerical Methods for Constrained Optimization

4.1 Production under an emissions cap

Two producers choose output q_1 and q_2 . Each earns profit

$$\pi_i(q_i) = 12q_i - q_i^2, \quad q_i \geq 0.$$

- Marginal profit is $12 - 2q_i$: additional production becomes less profitable as output increases.
- Producers have the same profit function but different pollution technologies:

$$e_1(q_1) = \frac{1}{2}q_1^2, \quad e_2(q_2) = q_2^2.$$

- Emissions increase at an increasing rate.
- At the same output, producer 2 generates twice as much pollution as producer 1.
- Think of different equipment or production processes, rather than uncertainty about outcomes.

What is the private optimum without regulation? What is the total profit and total emissions?

Without regulation, $q_1^u = q_2^u = 6$, total profit is 72, and total emissions are 54.

A regulator wants to achieve an aggregate emissions threshold E while preserving as much total profit as possible.

What is the regulators problem? What is the objective and what is the constraint?

$$\max_{q_1, q_2 \geq 0} \Pi(q) = 12q_1 - q_1^2 + 12q_2 - q_2^2 \quad \text{subject to} \quad G(q) = \frac{1}{2}q_1^2 + q_2^2 \leq E.$$

- This is a **cost-effective allocation for a given target**.
- We have not specified environmental damages, so the model does not determine the socially optimal target.
- Production reductions are the only way to reduce pollution in this example.

If $E < 54$, can both producers continue producing 6 units? Should the regulator require equal output from the two producers?

4.2 Theory: the KKT conditions

Using the same maximization convention as before, write

$$\mathcal{L}(q, \lambda, \eta) = \Pi(q) + \lambda[E - G(q)] + \eta_1 q_1 + \eta_2 q_2.$$

The KKT conditions are:

1. **Stationarity:**

$$12 - 2q_1 - \lambda q_1 + \eta_1 = 0,$$

$$12 - 2q_2 - 2\lambda q_2 + \eta_2 = 0.$$

2. **Primal feasibility:** $G(q) \leq E$ and $q_1, q_2 \geq 0$.

3. **Dual feasibility:** $\lambda, \eta_1, \eta_2 \geq 0$.

4. **Complementary slackness:**

$$\lambda[E - G(q)] = 0, \quad \eta_i q_i = 0.$$

For $0 < E < 54$, the optimal outputs are positive and the cap binds, so $\eta_1 = \eta_2 = 0$.

$$q_1(\lambda) = \frac{12}{2 + \lambda}, \quad q_2(\lambda) = \frac{12}{2 + 2\lambda}.$$

i Note

Recall that λ is the shadow price of the emissions cap. The KKT conditions imply

$$\frac{\pi'_1(q_1)}{e'_1(q_1)} = \frac{\pi'_2(q_2)}{e'_2(q_2)} = \lambda.$$

Each ratio measures **marginal profit sacrificed per unit of emissions reduction**. Reducing output by a small amount lowers both profit and emissions. If this ratio differs across producers, shift pollution reductions toward the producer that can reduce emissions at lower marginal cost.

Substitute these expressions into $G(q) = E$ to obtain one equation in λ (assuming an interior solution):

$$\frac{1}{2} \left(\frac{12}{2 + \lambda} \right)^2 + \left(\frac{12}{2 + 2\lambda} \right)^2 - E = 0.$$

Simplifying gives a nonlinear equation in λ (the emissions function is quadratic in output):

$$\frac{72}{(2 + \lambda)^2} + \frac{144}{(2 + 2\lambda)^2} - E = 0.$$

4.2.1 Finding the multiplier by rootfinding

Choose $E = 17$ and define the excess-emissions function

$$h(\lambda) = \frac{72}{(2 + \lambda)^2} + \frac{144}{(2 + 2\lambda)^2} - 17.$$

For any proposed shadow price λ , the stationarity equations determine outputs $q_1(\lambda)$ and $q_2(\lambda)$. The function $h(\lambda)$ measures how far their total emissions exceed the cap. We seek the price at which $h(\lambda) = 0$.

- At $\lambda = 0$, emissions are 54, so $h(0) = 37 > 0$.
- At $\lambda = 2$, emissions are 8.5, so $h(2) = -8.5 < 0$.
- As λ increases, both outputs and emissions decrease. Thus h is strictly decreasing for $\lambda \geq 0$, and there is exactly one root between 0 and 2.

We can use bisection, or R's `uniroot`, which combines bracketing with interpolation. Solving for λ first reduces the three KKT equations to one rootfinding problem.

```
h <- function(lambda) {
  0.5 * (12 / (2 + lambda))^2 +
  (12 / (2 + 2 * lambda))^2 - 17
}

lambda_root <- uniroot(h, interval = c(0, 2), tol = 1e-10)$root
q_root <- c(12 / (2 + lambda_root),
           12 / (2 + 2 * lambda_root))

c(lambda = lambda_root, q1 = q_root[1], q2 = q_root[2],
  excess_emissions = h(lambda_root))
```

lambda	q1	q2	excess_emissions
1.000000e+00	4.000000e+00	3.000000e+00	-2.440217e-10

The plot shows the root where excess emissions cross zero. Positive values indicate that the proposed price is too low to meet the cap; negative values indicate that emissions fall below the cap.

```
library(ggplot2)
lambda_grid <- data.frame(lambda = seq(0, 2, length.out = 201))
lambda_grid$excess <- h(lambda_grid$lambda)
root_plot <- ggplot(lambda_grid, aes(lambda, excess)) +
  geom_hline(yintercept = 0, colour = "grey40") +
  geom_line(colour = "steelblue", linewidth = 1) +
  geom_vline(xintercept = lambda_root, linetype = "dashed",
    colour = "firebrick") +
  annotate("point", x = lambda_root, y = 0,
    colour = "firebrick", size = 3) +
  annotate("text", x = lambda_root + 0.08, y = 5,
    label = "Root: lambda = 1", hjust = 0) +
  labs(x = expression(lambda),
    y = expression(h(lambda) == "Emissions minus cap")) +
  theme_minimal()
print(root_plot)
```

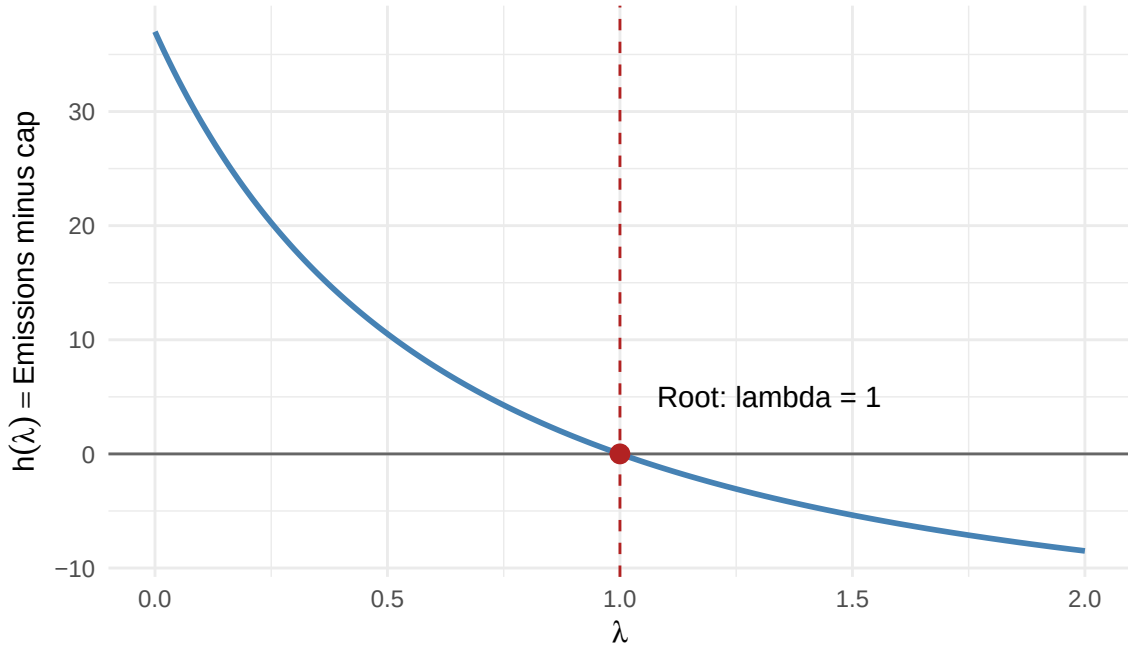


Figure 5: Rootfinding for the emissions-cap multiplier. At $\lambda = 1$, the implied outputs generate exactly 17 emissions units.

4.2.2 A transparent benchmark

The numerical root is $\lambda^* \approx 1$. Substituting the exact value $\lambda^* = 1$ verifies the solution:

$$q_1^* = 4, \quad q_2^* = 3, \quad G(q^*) = 8 + 9 = 17,$$

and total profit is

$$\Pi(q^*) = (48 - 16) + (36 - 9) = 59.$$

These values satisfy all the KKT conditions. The objective is strictly concave, and the feasible set is convex because G is convex. With $E > 0$, a strictly feasible positive allocation exists. The KKT solution is therefore the unique global maximum.

4.2.3 Geometry

The emissions boundary is part of an ellipse. Profit contours are circles centered at the unconstrained optimum (6, 6). The constrained optimum occurs where a profit contour is tangent to the emissions boundary:

$$\nabla \Pi(q^*) = \lambda^* \nabla G(q^*).$$

The objective gradient is **not zero**: it points toward higher profit outside the feasible set. There is no first-order profit gain along a direction tangent to the boundary.

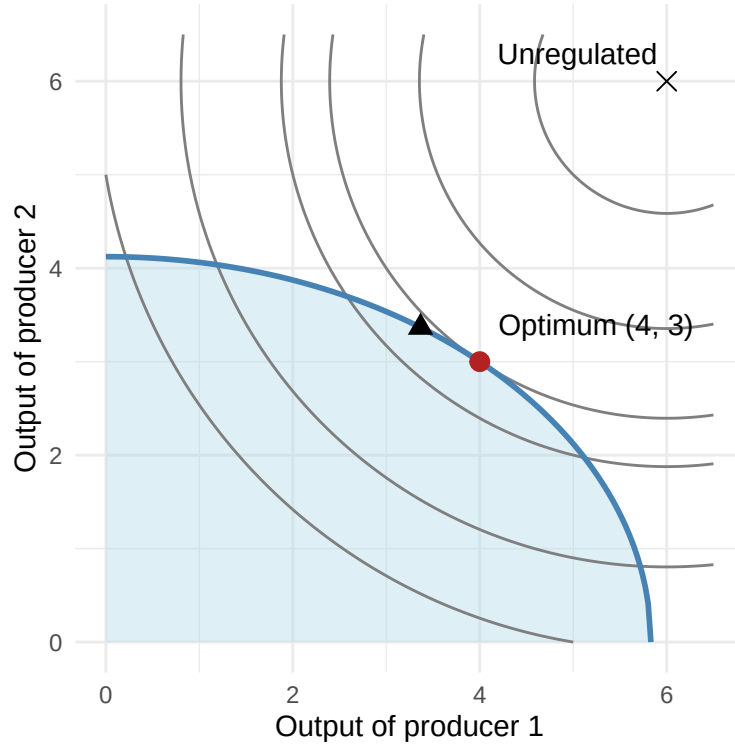


Figure 6: Profit contours and the emissions cap. The optimum is at (4, 3); equal output satisfying the same cap is less profitable.

4.2.4 Policy interpretation

Let $V(E)$ denote maximized total profit at cap E . The envelope theorem gives

$$V'(E) = \lambda^*.$$

At $E = 17$, relaxing the cap slightly raises maximized profit by approximately one monetary unit per additional emissions unit. Tightening it slightly lowers profit at the same marginal rate.

An emissions tax of $\tau = \lambda^* = 1$ per emissions unit decentralizes this allocation: each producer maximizes $\pi_i(q_i) - \tau e_i(q_i)$. Their first-order conditions match those above. A uniform emissions tax produces different marginal charges per unit of output because emissions responses differ.

Tax payments are transfers to the government. Compare resource costs using **pre-tax profit**, and distinguish that from producers' profit after paying taxes.

A common output limit satisfying the cap gives $q_1 = q_2 = \sqrt{17/1.5}$. Why might this achieve the target at a higher profit cost?

4.3 Why sequential quadratic programming?

Unconstrained gradient ascent follows $\nabla \Pi$. That direction can violate the emissions cap. Simply checking whether the objective gradient is small would also reject our correct solution, where $\nabla \Pi = (4, 6)$.

Sequential quadratic programming (SQP) combines the curvature information used by Newton's method with the feasibility and multiplier conditions used by constrained optimization. Each iteration solves a smaller quadratic program to propose the next step.

4.3.1 A local approximation

The original problem chooses the output vector $q = (q_1, q_2)^\top$. SQP instead asks a smaller question at each iteration: **starting from our current outputs, what change should we try next?**

4.3.1.1 Step 1: define the current point and the proposed change

For the algorithm, convert maximization to minimization:

$$F(q) = -\Pi(q), \quad c(q) = G(q) - E \leq 0.$$

Minimizing negative profit gives the same allocation as maximizing profit. Writing the constraint as emissions minus the cap makes its sign easy to interpret: negative is feasible, zero is binding, and positive violates the cap.

Symbol	Meaning
q^k	Current output vector at iteration k ; the superscript labels the iteration, not a power
$d = (d_1, d_2)^\top$	Proposed change in outputs; this is the decision variable of the subproblem
$q^k + d$	Proposed outputs if we take the full step
α_k	Fraction of the proposed step actually taken
B^k	Matrix describing the curvature used in the local model

A positive d_i increases producer i 's output; a negative d_i reduces it. We optimize over **changes** d , then translate the result back into outputs.

4.3.1.2 Step 2: approximate the objective near the current point

Recall the second-order Taylor expansion:

$$F(q^k + d) \approx F(q^k) + \nabla F(q^k)^\top d + \frac{1}{2} d^\top \nabla^2 F(q^k) d.$$

The three terms have different roles:

- $F(q^k)$ is the current objective value. It is constant as we vary d , so it can be omitted when choosing the step.
- $\nabla F(q^k)^\top d$ predicts the first-order change. A negative value predicts that negative profit falls, so profit rises.
- The quadratic term describes how marginal returns change as we move. It prevents the local model from treating an initially profitable direction as equally profitable at every distance.

For our objective,

$$F(q) = q_1^2 - 12q_1 + q_2^2 - 12q_2, \quad \nabla F(q) = \begin{bmatrix} 2q_1 - 12 \\ 2q_2 - 12 \end{bmatrix}, \quad \nabla^2 F(q) = \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix}.$$

Because F is quadratic, its second-order Taylor expansion is exact. For a general nonlinear objective, it is only a local approximation. This expansion uses the objective Hessian $\nabla^2 F$. Below, we will distinguish this Taylor model from the quadratic model used by SQP.

4.3.1.3 Step 3: approximate the constraint with a tangent line

Use a first-order Taylor expansion for the emissions constraint:

$$c(q^k + d) \approx c(q^k) + \nabla c(q^k)^\top d.$$

Thus, the local requirement is

$$c(q^k) + \nabla c(q^k)^\top d \leq 0.$$

For this example,

$$c(q) = \frac{1}{2}q_1^2 + q_2^2 - E, \quad \nabla c(q) = \begin{bmatrix} q_1 \\ 2q_2 \end{bmatrix}.$$

The current constraint value tells us how much slack we have, while the gradient predicts how a change in each output uses that slack. Geometrically, we replace the curved emissions boundary by a straight line near q^k . Nonnegativity is already linear, so it is imposed exactly as $q^k + d \geq 0$, or $d_i \geq -q_i^k$.

Why does $d_i \geq -q_i^k$ allow a negative step, even though output itself cannot be negative?

Because it could overshoot the boundary.

4.3.1.4 Step 4: combine the models into a quadratic program

A direct combination of the objective Taylor expansion and the linearized constraints would use $\nabla^2 F(q^k)$ as the quadratic coefficient. Standard SQP adjusts that coefficient to account for constraint curvature. Define its quadratic model as

$$m_k(d) = \nabla F(q^k)^\top d + \frac{1}{2} d^\top B^k d.$$

B^k is the **curvature matrix in the subproblem objective**. This describes where it is used; we still need to explain how SQP chooses it. The step solves

$$\min_d \nabla F(q^k)^\top d + \frac{1}{2} d^\top B^k d$$

subject to

$$c(q^k) + \nabla c(q^k)^\top d \leq 0, \quad q^k + d \geq 0.$$

This is a **quadratic program (QP)**: the objective is quadratic in d , and all constraints are linear in d . The gradient rewards promising changes, the curvature controls their size and balance, and the constraints restrict the proposed changes.

To choose B^k , SQP uses a Newton approximation to the **KKT conditions**, rather than only a Taylor approximation to F . For clarity, suppose the emissions constraint is active and the output bounds are inactive. Write

$$\mathcal{L}_{\min}(q, \lambda) = F(q) + \lambda c(q).$$

The original stationarity condition is

$$\nabla F(q) + \lambda \nabla c(q) = 0.$$

At the current estimates (q^k, λ^k) , propose changes d and $\Delta\lambda$. Linearizing stationarity in both variables gives

$$0 \approx \nabla F(q^k) + \lambda^k \nabla c(q^k) + \underbrace{[\nabla^2 F(q^k) + \lambda^k \nabla^2 c(q^k)]}_{B^k \text{ for an exact Newton step}} d + \nabla c(q^k) \Delta\lambda.$$

Define the proposed multiplier $\hat{\lambda} = \lambda^k + \Delta\lambda$. Combining the two multiplier terms gives

$$\nabla F(q^k) + B^k d + \hat{\lambda} \nabla c(q^k) = 0.$$

This is exactly the stationarity condition of the quadratic subproblem, because the gradient of $m_k(d)$ is $\nabla F(q^k) + B^k d$. Together with the linearized active constraint, it reproduces the Newton equations for the original KKT system. This explains the choice

$$B^k \approx \nabla_{qq}^2 \mathcal{L}_{\min}(q^k, \lambda^k) = \nabla^2 F(q^k) + \lambda^k \nabla^2 c(q^k).$$

Thus, B^k has one consistent meaning: **the quadratic coefficient in the SQP objective, chosen to approximate Lagrangian curvature**. The QP objective is a model for choosing a constrained step; when $\lambda^k \nabla^2 c \neq 0$, it is not the ordinary second-order Taylor expansion of F alone. The nonlinear constraint's curvature enters this coefficient because the QP replaces the constraint itself with a linear model.

For our example,

$$\nabla_{qq}^2 \mathcal{L}_{\min} = \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix} + \lambda \begin{bmatrix} 1 & 0 \\ 0 & 2 \end{bmatrix} = \begin{bmatrix} 2 + \lambda & 0 \\ 0 & 2 + 2\lambda \end{bmatrix}.$$

With linear constraints, $\nabla^2 c = 0$, so the Lagrangian and objective Hessians coincide. They also coincide at an iterate with $\lambda^k = 0$, which is the case in our illustrative first step below. In practice, SLSQP updates an approximation to Lagrangian curvature rather than asking us to supply this matrix.

4.3.1.5 Step 5: work through one numerical subproblem

Start at $q^0 = (2, 2)$ with $E = 17$. Total emissions are 6, giving 11 units of slack. For this illustrative first step, take $\lambda^0 = 0$ and $B^0 = 2I$, the objective Hessian. These are teaching choices, rather than a claim about SLSQP's internal initialization.

At this point,

$$\nabla F(q^0) = \begin{bmatrix} -8 \\ -8 \end{bmatrix}, \quad c(q^0) = -11, \quad \nabla c(q^0) = \begin{bmatrix} 2 \\ 4 \end{bmatrix}.$$

The subproblem becomes

$$\min_{d_1, d_2} -8d_1 - 8d_2 + d_1^2 + d_2^2$$

subject to

$$2d_1 + 4d_2 \leq 11, \quad d_1, d_2 \geq -2.$$

Without the emissions restriction, the objective first-order conditions give $d_1 = d_2 = 4$, moving directly to $(6, 6)$. But this step would use $2(4) + 4(4) = 24$ units of predicted slack, exceeding the available 11. The local emissions constraint must bind.

Let $\hat{\lambda}$ be its QP multiplier. With the output bounds inactive, the subproblem's stationarity conditions are

$$-8 + 2d_1 + 2\hat{\lambda} = 0, \quad -8 + 2d_2 + 4\hat{\lambda} = 0.$$

Thus $d_1 = 4 - \hat{\lambda}$ and $d_2 = 4 - 2\hat{\lambda}$. Substituting into the binding local constraint gives

$$2(4 - \hat{\lambda}) + 4(4 - 2\hat{\lambda}) = 11 \quad \Rightarrow \quad \hat{\lambda} = 1.3.$$

Therefore,

$$d^0 = (2.7, 1.4), \quad q^0 + d^0 = (4.7, 3.4).$$

The subproblem proposes a larger increase for the cleaner producer. Its multiplier is an estimate from the local problem, not yet the final shadow value $\lambda^* = 1$.

4.3.1.6 Step 6: check the proposed step against the original problem

The linearized constraint predicts that the full step exactly meets the cap. The actual emissions are

$$G(4.7, 3.4) = \frac{1}{2}(4.7)^2 + (3.4)^2 = 22.605 > 17.$$

The approximation omitted the quadratic change in emissions. In this example we can see that omission exactly:

$$c(q^k + d) = c(q^k) + \nabla c(q^k)^\top d + \frac{1}{2}d_1^2 + d_2^2.$$

For $d^0 = (2.7, 1.4)$, the omitted term is 5.605. The objective model is exact here, but the linearized constraint is not.

Taking half the proposed step instead gives

$$q^0 + \frac{1}{2}d^0 = (3.35, 2.7).$$

Allocation	Actual profit	Actual emissions	Feasible?
Current point (2, 2)	40	6	Yes
Full proposed step (4.7, 3.4)	63.55	22.605	No
Half step (3.35, 2.7)	54.0875	12.90125	Yes

The half step increases profit and remains feasible. It illustrates why we check the original functions and choose a step length; it does not prescribe the step length SLSQP will take. The next iteration constructs a new local model around the accepted point. **Sequential** means that we repeat this process, updating the point, derivatives, and curvature information as we go.

4.3.2 What each iteration does

1. Evaluate the objective, constraints, and their derivatives.
2. Solve the quadratic subproblem for a direction and multiplier estimates.
3. Select a step length and update $q^{k+1} = q^k + \alpha_k d^k$.
4. Update the curvature approximation and repeat.

A line search uses a **merit function**, such as $F(q) + \rho \max\{0, c(q)\}$, to balance objective improvement and constraint violation. The parameter $\rho > 0$ controls the penalty for violating the cap. Bounds also need to be respected.

Linearized feasibility does not guarantee feasibility for the original curved constraint. For example, at $q^k = (4, 3)$, $d = (0.3, -0.2)$ is tangent to the boundary because $(4, 6)^\top d = 0$, but

$$G(q^k + d) = 17 + \frac{1}{2}(0.3)^2 + (-0.2)^2 = 17.085.$$

This is why the solver must evaluate the actual nonlinear constraint again and control its step. Intermediate points can violate the cap even when the final solution satisfies it.

4.3.3 SLSQP and BFGS

We will use **Sequential Least Squares Quadratic Programming (SLSQP)**, an SQP implementation in NLOpt accessed through R's `nloptr` package. It handles smooth nonlinear equality and inequality constraints along with variable bounds.

SLSQP uses **BFGS** to update a curvature approximation from changes in points and gradients. This avoids requiring us to supply second derivatives. We provide first derivatives of the objective and constraint.

4.4 Numerical solution in R

The standalone script `emissions_cap.R` contains the same walkthrough for use in RStudio. Install `nloptr` once if needed: `install.packages("nloptr")`.

4.4.1 Step 1: define the economic model

```
profit <- function(q) sum(12*q - q^2)
profit_gradient <- function(q) 12 - 2*q
emissions <- function(q) 0.5*q[1]^2 + q[2]^2
emissions_gradient <- function(q) c(q[1], 2*q[2])
E <- 17

profit(c(6, 6))
```

[1] 72

```
emissions(c(6, 6))
```

[1] 54

4.4.2 Step 2: translate the model into the solver's conventions

nloptr minimizes its objective and expects inequality constraints in the form $c(q) \leq 0$. Negate profit and its gradient, and return emissions minus the cap.

```
objective <- function(q) -profit(q)
objective_gradient <- function(q) -profit_gradient(q)
cap_constraint <- function(q) emissions(q) - E
cap_jacobian <- function(q) matrix(emissions_gradient(q), nrow = 1)
```

The objective gradient has two entries. The constraint Jacobian has one row for the single constraint and two columns for the two decisions.

4.4.3 Step 3: solve

```
fit <- nloptr::nloptr(
  x0 = c(2, 2), # a feasible starting allocation
  eval_f = objective,
  eval_grad_f = objective_gradient,
  eval_g_ineq = cap_constraint,
  eval_jac_g_ineq = cap_jacobian,
  lb = c(0, 0),
  opts = list(
    algorithm = "NLOPT_LD_SLSQP",
    xtol_rel = 1e-9,
    ftol_abs = 1e-10,
    tol_constraints_ineq = 1e-9,
    maxeval = 500
  )
)
q_star <- fit$solution
fit$status
```

[1] 3

```
cat(paste(strwrap(fit$message, width = 65), collapse = "\n"), "\n")
```

NLOPT_FTOL_REACHED: Optimization stopped because ftol_rel or ftol_abs (above) was reached.

```
q_star

[1] 4 3

c(profit = profit(q_star), emissions = emissions(q_star),
  cap_residual = cap_constraint(q_star))

      profit      emissions cap_residual
5.900000e+01 1.700000e+01 4.263256e-14
```

The expected solution is approximately (4, 3), profit 59, and emissions 17. `xtol_rel` concerns changes in decisions; `ftol_abs` concerns changes in objective values. The constraint tolerance concerns feasibility. A stopping message is useful evidence, but it is not a substitute for checking the economic conditions. Reaching the evaluation limit is not proof of convergence.

4.4.4 Step 4: check the KKT conditions

Both outputs are positive, so their nonnegativity multipliers should be zero. Recover the cap multiplier from stationarity, using the least-squares ratio

$$\hat{\lambda} = \frac{\nabla G(q)^\top \nabla \Pi(q)}{\nabla G(q)^\top \nabla G(q)}.$$

Then check the remaining stationarity residuals explicitly. This ratio alone does not establish stationarity.

```
g_profit <- profit_gradient(q_star)
g_emissions <- emissions_gradient(q_star)
lambda_hat <- sum(g_emissions*g_profit) / sum(g_emissions^2)
checks <- c(
  cap_violation = max(0, cap_constraint(q_star)),
  bound_violation = max(0, -q_star),
  stationarity = max(abs(g_profit - lambda_hat*g_emissions)),
  dual_violation = max(0, -lambda_hat),
  complementarity = abs(lambda_hat*cap_constraint(q_star))
)
lambda_hat

[1] 1

data.frame(check = names(checks), residual = unname(checks))
```

```
      check      residual
1 cap_violation 4.263256e-14
2 bound_violation 0.000000e+00
3 stationarity 8.881784e-16
4 dual_violation 0.000000e+00
5 complementarity 4.263256e-14
```

All residuals should be close to zero and $\hat{\lambda}$ close to 1. This recovery assumes positive outputs and a binding cap; a slack cap has multiplier zero, and binding output bounds require their own multipliers.

4.5 Experiments: tighten the cap and compare policies

Wrap the solver in a function so we can change the target without changing the model.

```
solve_cap <- function(E, start = c(2, 2)) {
  fit <- nloptr::nloptr(
    x0 = start, eval_f = objective, eval_grad_f = objective_gradient,
    eval_g_ineq = function(q) emissions(q) - E,
    eval_jac_g_ineq = cap_jacobian, lb = c(0, 0),
    opts = list(algorithm = "NLOPT_LD_SLSQP", xtol_rel = 1e-9,
                ftol_abs = 1e-10, tol_constraints_ineq = 1e-9,
                maxeval = 500)
  )
  if (!(fit$status %in% 1:4) || emissions(fit$solution) > E + 1e-7) {
    stop("The solver did not return a converged, feasible solution.")
  }
  q <- fit$solution
  data.frame(cap = E, q1 = q[1], q2 = q[2],
             profit = profit(q), emissions = emissions(q))
}
```

```
delta <- 0.01
baseline <- solve_cap(17)
tighter <- solve_cap(17 - delta)
looser <- solve_cap(17 + delta)
rbind(tighter, baseline, looser)
```

	cap	q1	q2	profit	emissions
1	16.99	3.99907	2.998953	58.99	16.99
2	17.00	4.00000	3.000000	59.00	17.00
3	17.01	4.00093	3.001046	59.01	17.01

```
# Compare the numerical marginal value with lambda_hat.
(looser$profit - tighter$profit) / (2*delta)
```

```
[1] 1
```

```
# Equal output (also an equal percentage cut from the baseline 6, 6).
q_equal <- rep(sqrt(17/1.5), 2)
data.frame(
  policy = c("Efficient allocation", "Equal output restriction"),
  q1 = c(q_star[1], q_equal[1]),
  q2 = c(q_star[2], q_equal[2]),
  emissions = c(emissions(q_star), emissions(q_equal)),
  profit = c(profit(q_star), profit(q_equal)),
  profit_loss = 72 - c(profit(q_star), profit(q_equal))
)
```

	policy	q1	q2	emissions	profit	profit_loss
1	Efficient allocation	4.000000	3.000000	17	59.00000	13.00000
2	Equal output restriction	3.366502	3.366502	17	58.12937	13.87063

At the same emissions target, equal output yields profit of approximately 58.13, compared with 59 under the efficient allocation.

Try $E = 10$, $E = 30$, and $E = 60$. Predict whether the cap binds, how output changes, and whether the shadow value rises or falls. When the cap is slack, why should its multiplier be zero?

Restart the $E = 17$ problem at $(1, 1)$ and $(5, 5)$. The second start violates the cap. Do you recover the same solution? Here our concavity argument establishes global optimality; agreement across starts alone would not establish it in a general nonconvex problem.

4.6 Takeaways

- The target defines feasibility; the objective determines the best feasible allocation.
- At a constrained optimum, check the Lagrangian gradient rather than requiring the objective gradient to vanish.
- SQP repeatedly solves local quadratic programs with linearized constraints.
- Verify feasibility, stationarity, multiplier signs, and complementary slackness after solving.
- Interpret the multiplier as the marginal cost of tightening the target, and compare policies at the same target.

Method references: [NLOpt's SLSQP documentation](#) and [R's optimization documentation](#).